

Diseño de la Arquitectura del Sistema

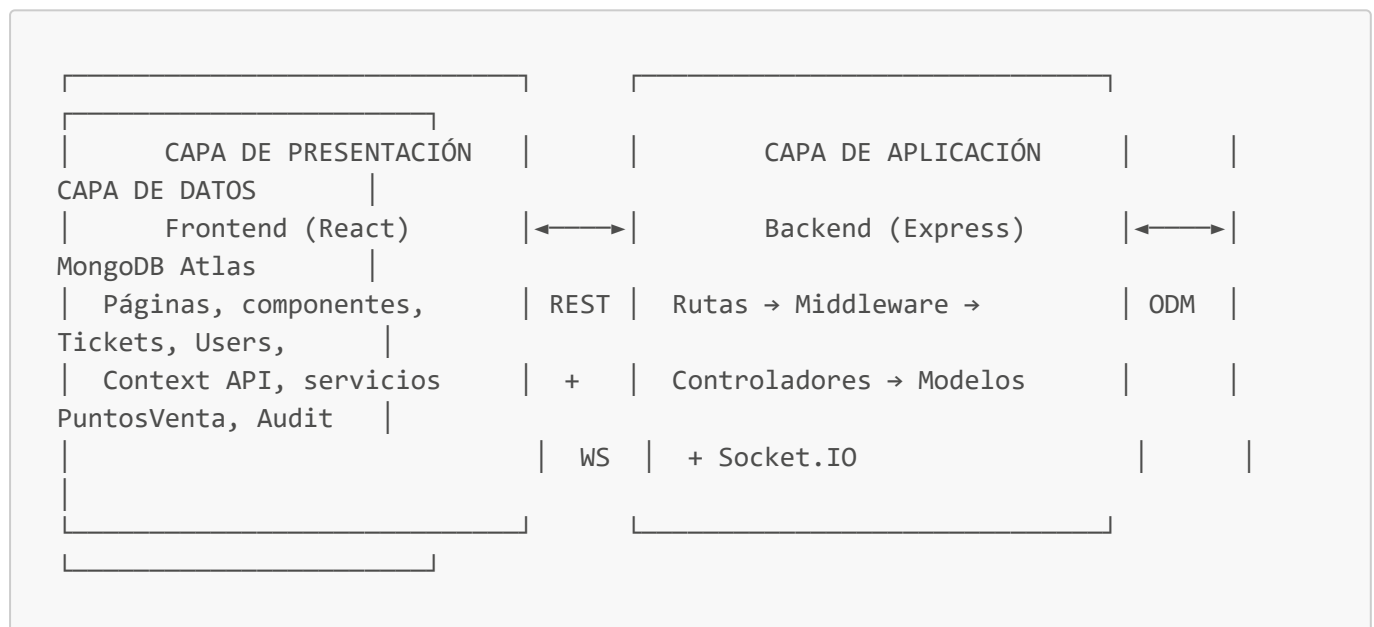
Actividad N°: 11 **Fecha:** 15/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Diseñar la arquitectura general del sistema a partir del stack y las decisiones técnicas aprobadas en la Semana 2, definiendo capas, patrones y flujo de comunicación entre frontend, backend y base de datos.

2. Estilo arquitectónico

Se adopta una arquitectura **cliente-servidor de 3 capas**, con una API REST como contrato principal y un canal complementario de WebSocket para eventos en tiempo real:



3. Patrón interno del backend (estilo MVC adaptado)

- **Routes** (`src/routes/*.js`): definen los endpoints y encadenan los middlewares correspondientes.
- **Middleware** (`src/middleware/*.js`): resuelven responsabilidades transversales antes de llegar al controlador:
 - `auth`: valida el JWT y adjunta `req.user`.
 - `authorize(...roles)`: valida permisos por rol.
 - `selectCollection`: adjunta `req.TicketModel` según la colección activa.
 - `auditLogger`: registra automáticamente ciertas operaciones en el log de auditoría.
- **Controllers** (`src/controllers/*.js`): contienen la lógica de negocio (validaciones, reglas, respuesta HTTP).
- **Models** (`src/models/*.js`): esquemas Mongoose que representan las colecciones de MongoDB.

Este patrón permite que cada endpoint sea una composición explícita de middlewares reutilizables, por ejemplo:

```
router.post('/:id/canje', auth, authorize('jefe', 'staff'), auditLogger('canje'),  
  canjeTicket);
```

4. Patrón en el frontend

- **Pages:** una página por vista funcional (Dashboard, TicketsPage, UsersPage, AuditPage, PuntosVenta, Login, ChangePassword).
- **Components:** elementos reutilizables entre páginas (Layout, Navigation, ProtectedRoute, RoleBasedRedirect, Unauthorized).
- **Context API (AuthContext):** fuente única de verdad del usuario autenticado y su token, evita prop-drilling.
- **Services:** capa de acceso a datos (`api.js` para REST vía Axios, `socket.js` como singleton de conexión WebSocket).

5. Comunicación en tiempo real (diseño de salas)

Se diseñan **salas (rooms) de Socket.IO** para aislar la información según el contexto del usuario:

```
punto-venta-{idPuntoVenta}    → usuarios Jefe siguiendo un punto de venta  
específico  
staff-{nombrePuntoTrabajo}    → usuarios Staff de un punto de trabajo específico
```

Cuando ocurre un canje o impresión, el backend emite el evento `ticket-updated` únicamente a las salas relevantes, evitando que un usuario reciba datos de puntos de venta ajenos (principio de necesidad de conocimiento, alineado con RNF-03).

6. Alineación con requerimientos no funcionales

Decisión de arquitectura	RNF que soporta
Middleware <code>auth/authorize</code> en cada ruta sensible	RNF-01, RNF-03
Salas de Socket.IO aisladas por punto de venta/trabajo	RNF-03 (control de acceso), RF-13
Índices en MongoDB sobre campos de búsqueda frecuente	RNF-07 (rendimiento)
Middleware de auditoría desacoplado del controlador	RNF-08 (trazabilidad) sin acoplar lógica de negocio
Despliegue independiente de frontend/backend	RNF-10

7. Conclusiones del día

Se define una arquitectura de 3 capas con un patrón de middlewares componibles en el backend y una separación clara de responsabilidades en el frontend, que servirá de base para el diseño detallado de la base de datos (día siguiente) y de los módulos de software.

Observaciones: Sin observaciones.

Diseño de la Base de Datos MongoDB

Actividad N°: 12 **Fecha:** 16/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Diseñar el modelo de datos en MongoDB: colecciones, campos, relaciones (referencias) e índices, alineado con los requerimientos funcionales y de rendimiento definidos previamente.

2. Modelo de colecciones



```
timestamps (createdAt, updatedAt)
```

3. Justificación de diseño por colección

Ticket

- Los nombres de campo replican literalmente las columnas del CSV de origen (**First Name**, **Seat**, **Ticket ID**, etc.) para simplificar la importación directa sin necesidad de mapear/renombrar cada columna.
- **Ticket ID** es **unique** e indexado: es la clave natural de negocio para evitar boletos duplicados en reimportaciones.
- Los campos de canje (**canjeado**, **fechaCanje**, **usuarioCanje**, **puntoCanje**) están separados de los campos de impresión (**impreso**, **fechaImpresion**, **usuarioResponsable**) para permitir distinguir ambos eventos en el ciclo de vida del boleto.
- **reimpresiones** se modela como subdocumento embebido (arreglo), ya que su ciclo de vida depende completamente del ticket padre y no requiere consultarse de forma independiente.

User

- **puntoTrabajo** es condicionalmente requerido solo si **rol** = **'staff'**, reflejando que un Jefe no está atado a un único punto físico.
- **creadoPor** es una referencia a sí misma (self-reference), permitiendo trazar qué Jefe creó cada cuenta.
- La contraseña nunca se expone: se hashea en un hook **pre('save')** y se excluye explícitamente en **toJSON()**.

PuntoVenta

- **localidades** es un arreglo de strings **sin enum fijo**, por diseño: el valor proviene de datos reales del evento (columna Seat del CSV) y cambia de un concierto a otro.

AuditLog

- **tipo** sí usa **enum** fijo, porque los tipos de operación auditable son controlados por el propio sistema (no por datos externos variables como las localidades).
- **detalles** usa **Mixed** para admitir distinta forma de datos según el tipo de operación (ej. un canje individual vs. un canje masivo con contador de tickets).

4. Diseño de índices (rendimiento)

Colección	Índice	Propósito
Ticket	Ticket ID (unique)	Búsqueda directa y prevención de duplicados
Ticket	First Name + Last Name	Búsqueda por nombre completo
Ticket	Seat	Filtro por localidad
Ticket	Numero de Cedula	Búsqueda por cédula

Colección	Índice	Propósito
Ticket	<code>impreso + puntoTrabajo, canjeado + puntoTrabajo</code>	Filtros combinados frecuentes en la operación diaria
Ticket	<code>updatedAt (desc), Ticket + updatedAt</code>	Sincronización/verificación de cambios recientes
PuntoVenta	<code>nombre, activo</code>	Listado y validación de unicidad
AuditLog	<code>tipo + createdAt, usuario + createdAt, ticketId + tipo</code>	Consultas de auditoría por tipo, por usuario o por boleto

5. Relaciones entre colecciones

- `Ticket.usuarioResponsable, Ticket.usuarioCanje, Ticket.reimpresiones[].usuario` → referencian `User._id`.
- `PuntoVenta.creadoPor` → referencia `User._id`.
- `AuditLog.usuario` → referencia `User._id`.
- No existe relación directa `Ticket ↔ PuntoVenta` por `_id`; la asociación es lógica, a través del valor de `Seat/puntoTrabajo` (string), lo que evita depender de un ID fijo cuando cambian las localidades entre eventos.

6. Conclusiones del día

El modelo de datos queda diseñado en 4 colecciones con relaciones claras, priorizando búsquedas rápidas mediante índices y manteniendo flexible el campo de localidades para no atar el sistema a un evento específico.

Observaciones: Sin observaciones.

Diseño de Módulos y Componentes de Software

Actividad N°: 13 **Fecha:** 17/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

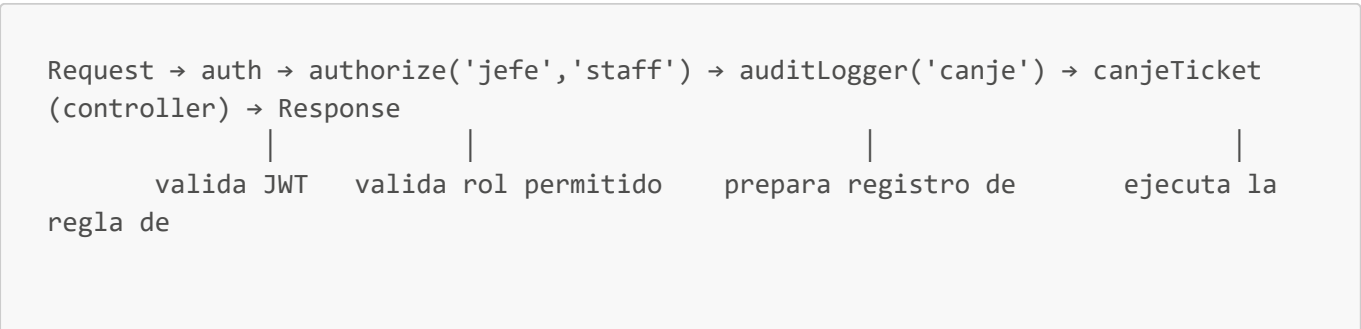
Definir la organización modular del código, tanto en el backend como en el frontend, especificando la responsabilidad de cada módulo y cómo se relacionan entre sí.

2. Módulos del Backend

Módulo	Carpeta	Responsabilidad
Configuración	src/config/	Conexión a MongoDB (<code>database.js</code>), logger (<code>logger.js</code>), resolución de la colección activa de tickets (<code>collections.js</code>)
Modelos	src/models/	Esquemas Mongoose: <code>Ticket</code> , <code>User</code> , <code>PuntoVenta</code> , <code>AuditLog</code>
Controladores	src/controllers/	Lógica de negocio: <code>ticketController</code> , <code>userController</code> , <code>puntoVentaController</code> , <code>auditController</code> , <code>authController</code>
Rutas	src/routes/	Definición de endpoints y composición de middlewares por ruta
Middleware	src/middleware/	<code>auth</code> (JWT), <code>auditLogger</code> (registro automático), <code>collectionSelector</code> (colección activa), <code>cache</code> (respuestas cacheadas)
Scripts	src/scripts/	Utilidades operativas: <code>setup.js</code> (importación completa), <code>importCSV.js</code> , <code>extractLocalidades.js</code> , <code>createAdmin.js</code> , <code>createIndexes.js</code>
Utilidades	src/utils/	Funciones auxiliares: <code>csvImporter.js</code> , <code>ecuadorTime.js</code> (manejo de zona horaria), <code>logger.js</code> , <code>auditLogger.js</code>
Entrada de la app	src/app.js	Configuración de Express, seguridad, CORS, Socket.IO y arranque del servidor

3. Cadena de middlewares por endpoint (diseño de composición)

Ejemplo real de la ruta de canje (`routes/tickets.js`):



responde	auditoría	negocio y
----------	-----------	-----------

Este diseño permite **reordenar o reutilizar** middlewares sin duplicar lógica: por ejemplo, `auth` y `authorize` se reutilizan en todas las rutas protegidas del sistema (tickets, users, audit, puntos-venta).

4. Módulos del Frontend

Módulo	Carpeta	Responsabilidad
Páginas	<code>src/pages/</code>	Una vista funcional por página: <code>Login</code> , <code>Dashboard</code> , <code>TicketsPage</code> , <code>UsersPage</code> , <code>AuditPage</code> , <code>PuntosVenta</code> , <code>ChangePassword</code>
Componentes	<code>src/components/</code>	<code>Layout</code> (envoltura general), <code>Navigation</code> (barra superior), <code>ProtectedRoute</code> (guard de rutas por rol), <code>RoleBasedRedirect</code> (redirección según rol tras login), <code>Unauthorized</code>
Contexto	<code>src/context/AuthContext.jsx</code>	Estado global de sesión: usuario autenticado, token, funciones de login/logout
Servicios	<code>src/services/</code>	<code>api.js</code> (cliente Axios con interceptores), <code>socket.js</code> (cliente Socket.IO como singleton), <code>index.js</code> (agregador)
Estilos	<code>src/styles/theme.css</code>	Paleta y estilos de marca (colores FTT)

5. Componente `ProtectedRoute` (diseño de control de acceso en frontend)

Actúa como una guarda declarativa alrededor de cada página sensible:

```
<Route path="/dashboard" element={
  <ProtectedRoute roles={['jefe']}>
    <Dashboard />
  </ProtectedRoute>
} />
```

Responsabilidad: verificar que exista sesión activa y que el rol del usuario esté incluido en `roles`; si no cumple, redirige a `/login` o `/unauthorized` según corresponda. Este control es complementario al `authorize()` del backend — **la validación real y definitiva ocurre siempre en el servidor**, el frontend solo mejora la experiencia de usuario ocultando opciones no permitidas.

6. Servicio `socket.js` (patrón Singleton)

Se diseña como una única instancia (`socketService`) compartida por toda la aplicación, con responsabilidad de:

- Establecer la conexión autenticada con token.
- Unirse a salas (`joinPuntoVenta`, `joinStaff`) según el rol.
- Registrar/limpiar listeners de eventos (`ticket-updated`) evitando fugas de memoria al desmontar componentes.

7. Matriz de trazabilidad módulo → requerimiento

Requerimiento	Módulo(s) responsables
RF-01 (login)	<code>authController</code> , <code>middleware/auth</code> , <code>AuthContext</code> , <code>Login.jsx</code>
RF-02/RF-03 (búsqueda/filtro)	<code>ticketController.getTickets</code> , <code>TicketsPage.jsx</code>
RF-04/RF-05 (canje individual)	<code>ticketController.canjeTicket</code> , modelo <code>Ticket</code>
RF-06 (canje masivo)	<code>ticketController.bulkCanjeTickets</code> (<code>bulkWrite</code>)
RF-07 (reimpresión)	<code>ticketController.reprintTicket</code>
RF-08 (auditoría)	<code>middleware/auditLogger</code> , <code>auditController</code> , modelo <code>AuditLog</code>
RF-09 (usuarios)	<code>UserController</code> , <code>UsersPage.jsx</code>
RF-10 (puntos de venta)	<code>puntoVentaController</code> , <code>PuntosVenta.jsx</code>
RF-11 (dashboard)	<code>ticketController.getTicketStats</code> , <code>Dashboard.jsx</code>
RF-12 (importación CSV)	<code>scripts/setup.js</code> , <code>scripts/importCSV.js</code> , <code>utils/csvImporter.js</code>
RF-13 (tiempo real)	Socket.IO en <code>app.js</code> , <code>services/socket.js</code>

8. Conclusiones del día

Se documentó la organización modular completa del sistema y su matriz de trazabilidad con los requerimientos, evidenciando que cada requerimiento aprobado tiene un módulo concreto responsable de implementarlo.

Observaciones: Sin observaciones.

Diseño de Interfaces y Navegación

Actividad N°: 14 **Fecha:** 18/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

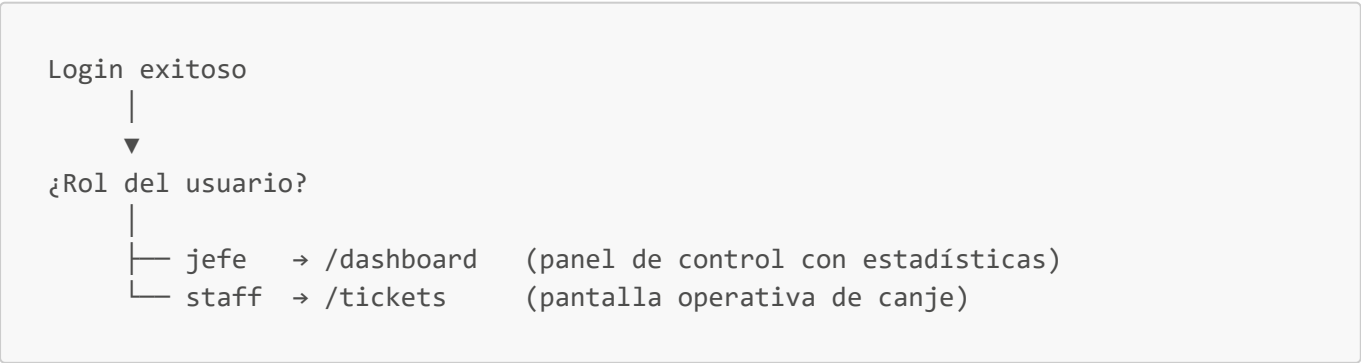
Diseñar el mapa de navegación del sistema y la estructura general de cada pantalla, diferenciando lo que ve un usuario **Jefe** de lo que ve un usuario **Staff**.

2. Mapa de rutas y control de acceso

Ruta	Acceso	Página
/login	Público	Login
/	Autenticado	Redirección automática según rol (RoleBasedRedirect)
/dashboard	Jefe	Dashboard
/puntos-venta	Jefe	PuntosVenta
/tickets	Jefe, Staff	TicketsPage
/users	Jefe	UsersPage
/audit	Jefe	AuditPage
/change-password	Autenticado (cualquier rol)	ChangePassword
/unauthorized	Autenticado	Página de acceso denegado
* (cualquier otra)	—	Redirige a /

3. Diseño de redirección post-login

Se diseña un componente **RoleBasedRedirect** que decide automáticamente a dónde enviar al usuario justo después de autenticarse, evitando que cada rol tenga que "saber" a qué URL ir manualmente:



4. Diseño de protección de rutas (dos niveles)

Nivel 1 – Frontend (experiencia de usuario)

ProtectedRoute verifica:

- a) ¿Hay sesión activa? → si no, redirige a /login
- b) ¿El rol del usuario está en la lista de roles permitidos de la ruta? → si no, redirige a /unauthorized

Nivel 2 – Backend (seguridad real)

authorize('jefe', ...) en cada endpoint valida el rol nuevamente antes de ejecutar cualquier operación (el frontend nunca es la única barrera)

5. Navegación visible según rol (barra superior)

Elemento de menú	Jefe	Staff
Dashboard	✓	✗
Puntos de Venta	✓	✗
Usuarios	✓	✗
Auditoría	✓	✗
Tickets	✓	✓
Cambiar Contraseña (menú de usuario)	✓	✓
Cerrar Sesión	✓	✓

Diseño de la barra: logo de marca (FeelTheTickets) a la izquierda, enlaces de navegación condicionados por rol al centro, y menú desplegable del usuario (nombre + badge de rol) a la derecha.

6. Estructura general de cada pantalla (wireframe funcional)

Login

- Logo FTT, formulario (usuario, contraseña), mensaje de contraseña por defecto para primer acceso.

Dashboard (Jefe)

- Tarjetas de resumen: total de boletos, canjeados, pendientes, % de avance.
- Gráfico de evolución diaria de canjes (Chart.js).
- Gráfico/tabla de distribución de canjes por punto de trabajo.
- Filtros por punto de trabajo y rango de fechas.

TicketsPage (Jefe, Staff)

- Selector de punto de venta/trabajo (si aplica).
- Barra de búsqueda (nombre, cédula, email, Ticket ID) + filtro por localidad.
- Tabla de boletos con columnas: Nombre, Localidad, Ticket ID, Estado (canjeado/pendiente), Acciones.

- Columna de checkboxes y botón "Canjear Seleccionados (n)" — visible según el rol autorizado para canje masivo.
- Modal de canje individual y modal de canje masivo (formulario: quién retira, parentesco si Otro, celular).
- Indicador de estado de conexión en tiempo real (Socket.IO).

PuntosVenta (Jefe)

- Listado de puntos de venta con sus localidades asociadas.
- Formulario de creación/edición (nombre, descripción, localidades detectadas del CSV).

UsersPage (Jefe)

- Listado de usuarios con rol y punto de trabajo.
- Formulario de creación/edición de usuario, con campo de punto de trabajo condicional (solo si rol = staff).

AuditPage (Jefe)

- Tabla de registros de auditoría: tipo de operación, usuario, ticket, punto de trabajo, fecha.
- Filtros por tipo de operación y usuario.

ChangePassword (cualquier rol autenticado)

- Formulario de cambio de contraseña, obligatorio en el primer acceso (`primerAcceso: true`).

7. Conclusiones del día

Se diseñó el mapa completo de navegación y la estructura funcional de cada pantalla, dejando explícita la diferenciación de experiencia entre los roles Jefe y Staff, y confirmando que la protección de rutas se refuerza en dos niveles (frontend y backend).

Observaciones: Sin observaciones.

Acta de Revisión del Diseño General

Actividad N°: 15 **Fecha:** 19/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo de la reunión

Revisar de forma integral el diseño producido durante la semana (arquitectura, base de datos, módulos e interfaces) antes de iniciar el desarrollo, verificando consistencia entre las decisiones de diseño y detectando posibles ajustes pendientes.

2. Participantes

Nombre	Rol
Miguel Vivanco	Tutor Empresarial (FeelTheTickets)
Gabriel	Pasante / Desarrollador

3. Documentos presentados

- [11_Disenio_Arquitectura_Sistema.md](#)
- [12_Disenio_Base_Datos_MongoDB.md](#)
- [13_Disenio_Modulos_Componentes.md](#)
- [14_Disenio_Interfaces_Navegacion.md](#)

4. Puntos revisados y resultado

Punto revisado	Resultado
Arquitectura de 3 capas + Socket.IO por salas	✓ Aprobada
Modelo de datos (4 colecciones, índices, relaciones)	✓ Aprobado
Organización modular backend/frontend	✓ Aprobada
Matriz de trazabilidad módulo → requerimiento	✓ Aprobada, sin brechas de cobertura
Mapa de navegación y protección de rutas en 2 niveles	✓ Aprobado

5. Hallazgo detectado durante la revisión

Al revisar en conjunto el diseño de interfaces (menú visible solo para Jefe) contra el diseño de rutas del backend, se detectó una **inconsistencia de permisos** en el canje masivo:

- El **frontend** (menú y checkboxes de selección múltiple) solo lo muestra al rol **Jefe**.
- El **backend** ([POST /api/tickets/bulk-canje](#)) actualmente autoriza tanto a **Jefe como a Staff** a nivel de API.

Decisión del tutor empresarial: se mantiene la restricción de negocio de que el canje masivo es una operación de supervisión y debe quedar exclusivamente para el rol Jefe. Se registra como **acción pendiente para la etapa de desarrollo/control de acceso (Semana 4)**: ajustar el backend para que `authorize('jefe')` sea el único rol permitido en la ruta `bulk-canje`, alineando el backend con el diseño de interfaz ya aprobado.

6. Acuerdos y siguientes pasos

1. Se aprueba el diseño general del sistema (arquitectura, base de datos, módulos e interfaces).
2. Se registra como pendiente técnico la corrección de permisos de `bulk-canje` para la Semana 4 (módulo de autenticación, usuarios y control de acceso).
3. Se autoriza iniciar la Semana 4: desarrollo de los módulos de autenticación, gestión de usuarios y control de acceso al sistema.

Firma Pasante: _____ **Firma Tutor Empresarial:** _____

Observaciones generales: Se detectó y documentó una inconsistencia de permisos entre frontend y backend en el canje masivo, quedando como pendiente de ajuste en la siguiente etapa.